Memory has Many Faces: Simplicial Complexes as an Indexing Layer for Agent Memory

Luke Tandjung

***Abstract*—This paper proposes simplicial complexes as a high-order representation of the agent memory layer.**

***Index Terms*—Agent Memory, Simplicial Complexes, Knowledge Graphs, RAG**

I. Introduction

Current state-of-the-art agent memory architectures vary in data modelling approaches for storing chunked text. Some use memories linked together in relational knowledge chains for hierarchical retrieval of raw text chunks [1]. Others create separate semantic memory graphs for epistemic categorisation, with typed edges [2]. Finally, some choose not to have any indexing structure at all [3]. At their core, these approaches aim to allow relationships between different text chunks to be expressed, while retaining powerful semantic similarity search features. We argue that this type of relationship modelling – representing relationships as binary edge interactions between memories – is representationally incomplete. Robust agent memory architecture must also capture co-occurrence.

Here, co-occurrence refers to the joint relationship that arises between more than two entities in the same context [4]. Some examples of it are multiple topics or events referenced in a single chat message, multiple people in a group conversation, or multiple events happening closely in time or space. When projecting co-occurrence onto semantic graph structures, information about joint relationships is lost [4].

Most agent memory architectures handle this implicitly. Observed approaches include injecting raw text chunks to preserve chat session co-occurrence [1], [3], storing time-of-event fields in memories for temporal co-occurrence [1], and using typed graph edges for semantic, temporal, and causal co-occurrence [2]. In this paper, we propose simplicial complexes in the form of simplex trees as an indexing structure for co-occurrence. Rather than fully relying on raw text chunks or constructing hierarchical indexes, simplex trees capture observed co-occurrence directly, enabling complete retrieval without dynamic graph traversal. We begin by motivating the choice of simplicial complexes through the lens of category theory. Readers primarily interested in the practical structure of simplicial complexes may skip to section 3.

II. Motivation: A Category Theoretic Lens

Category. A category C consists of

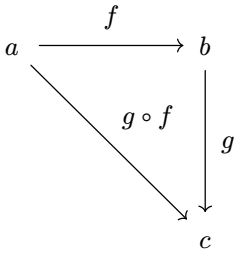
- A collection of objects $a, b, c, ...$ denoted formally as the **class** $\mathrm{ob}(C)$.
- A collection of **morphisms** (arrows) $\hom_C(a, b)$ for each object pair a, b in $\mathrm{ob}(C)$. The expression $f : a \to b$ indicates that f is a morphism that maps a to b, as depicted in the commutative diagram below. The collection of $\hom_C(a, b)$ for all object pairs a, b in $\mathrm{ob}(C)$ is denoted as $\hom(C)$.

$$a \xrightarrow{\quad f \quad} b$$

- The composition binary operator $\circ$; that is, for any three objects a, b, c in $\mathrm{ob}(C)$,

$$\circ : \hom_C(a, b) \times \hom_C(b, c) \to \hom_C(a, b) \quad (1)$$

That is, given morphisms $f : a \to b$ and $g : b \to c$, the morphisms $g \circ f : a \to c$ exists, depicted below.



Furthermore, the composition operation satisfies **associativity** and **identity** rules.

1) **Associativity**: For four objects a, b, c, d in $\mathrm{ob}(C)$ and morphisms $f : a \to b,\ g : b \to c,\ h : c \to d$, we have

$$(h \circ g) \circ f = h \circ (g \circ f) \quad (2)$$

2) **Identity**: For every object x in $\mathrm{ob}(C)$, there exists an identity morphism $\mathrm{id}_x : x \to x$ such that for every morphism $f : a \to b$, we have

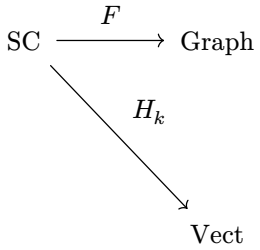
$$\mathrm{id}_b \circ f = f = f \circ \mathrm{id}_a \quad (3)$$

Some examples of categories and their morphisms are sets and functions, groups and group homeomorphisms, vector spaces and linear maps, and topologies and continuous maps.

Functor. A functor $F : C \to D$ is a **structure-preserving** map between categories C and D. In particular,

- it assigns each object a in $\mathrm{ob}(C)$ an object $F(a)$ in $\mathrm{ob}(D)$.
- it assigns each morphism $f : a \to b$ in $\mathrm{hom}(C)$ a morphism $F(f) : F(a) \to F(b)$ in $\mathrm{hom}(D)$, such that $F(\mathrm{id}_x) = \mathrm{id}_{F(x)}$ and $F(f \circ g) = F(f) \circ F(g)$.

With that in mind, we can view representations as categories: vector embeddings as **Vect**, directed labelled graphs as **Vect**, and simplicial complexes as **SC**. Mapping from simplicial complexes to graphs happens via the **1-skeleton functor** F. Likewise, mapping from simplicial complexes to vector spaces happens via the **simplicial homology functor** H_k.



However, as seen from the above diagram, no natural functor exists from graphs back to simplicial complexes. Given three mutually connected entities $\{a, b, c\}$, the graph doesn't indicate whether these entities co-occurred together (a filled triangle) or merely pairwise across contexts (an empty triangle) as seen in Fig. 1. This presents an implication for agent memory, When users discuss multiple concepts together, these co-occurrences carry semantic significance beyond pairwise projections. Graph-based systems either discard this at storage or attempt reconstruction at retrieval, both degrading performance.

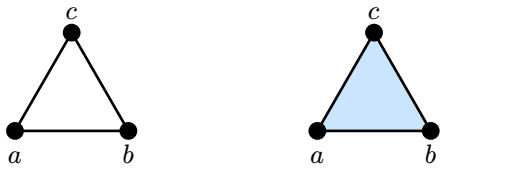


Fig. 1. A graph represents only pairwise edges, while a simplicial complex captures the 2-simplex $\{a, b, c\}$ as a filled face, encoding joint co-occurrence.

Simplicial complexes avoid this by representing higher-order co-occurences as first class objects. The functor hierarchy establishes representational power: complexes project to graphs and to embeddings, but systems at lower levels cannot recover richer structure. This points towards how memory systems should store information in the most structured form and project to coarser representations only when required.

III. Simplicial Complex for Knowledge

Simplicial Complex. A simplicial complex (Fig. 2) is a pair $R = (V, S)$ where

- V is the **vertex set** $V = \{v_1, v_2, ..., v_n\}$
- S is the **simplex/face set**. It is the set of non-empty subsets of V that satisfies the following properties by construction:
  1) $v \in V \Rightarrow \{v\} \in S$
  2) $s_1 \in S, s_2 \subseteq s_1 \Rightarrow s_2 \in S$. Here, s_2 is called the **face** of the simplex s_1. Furthermore, if $s_2 \subset s_1$, s_2 is called the **proper face** of the simplex s_1.

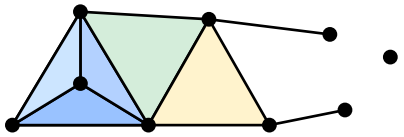


Fig. 2. A simplicial complex containing a 3-simplex (blue tetrahedron), 2-simplices (filled triangles), 1-simplices (edges), and 0-simplices (vertices). By downward closure, all faces of higher-dimensional simplices are automatically included.

The second property of **downward closure** for simplicial complexes distinguishes simplicial complexes from other possible candidates of knowledge representation like hypergraphs. If entities $\{v_1, v_2, v_3\}$ exists, then $\{v_1, v_2\}$, $\{v_2, v_3\}$, $\{v_1, v_3\}$, $\{v_1\}$, $\{v_2\}$, $\{v_3\}$ must exist. This captures semantic intuition: joint co-occurrence implies all sub-co-occurrences occurred. If three concepts were discussed together, each pair was discussed, and each individually. The way to describe such co-occurence is baked into the structure of simplicial complexes. If $s \in S$ has $k + 1$ elements, where $k \geq 0$, s is said to be a ***k*-simplex** of dimension k. A point is a 0-simplex, a line a 1-simplex, a filled triangle a 2-simplex, and a tetrahedron a 3-simplex (Fig. 3).

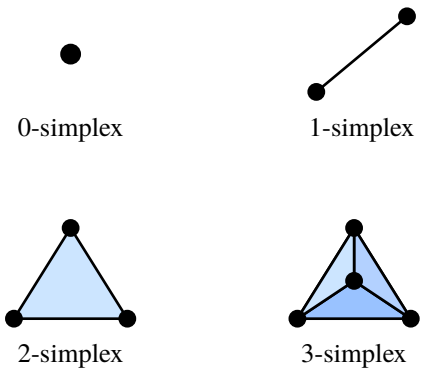


Fig. 3. Simplices of dimension 0 through 3: a vertex, edge, filled triangle, and tetrahedron.

For agent memory, this aligns with how knowledge emerges from contexts. A chat message referencing entities "neural networks", "backpropagation", and "gradient descent" produces a 2-simplex. By downward closure, this implies the existence of co-occurence between each entity pair as 1-simplex. This idea is generalised in the structure of the simplicial complex through **facets**, which are the maximal dimension proper faces

of a simplex. This involves taking the possible combinations of k points of a k-simplex; a 3-simplex $s = \{v_1, v_2, v_3, v_4\}$ will have the facets $\{v_1, v_2, v_3\}$, $\{v_2, v_3, v_4\}$, $\{v_1, v_3, v_4\}$, $\{v_1, v_2, v_4\}$. At the same time, the same simplex itself could be contained in another higher-dimensional simplices representing deeper structural pattern. Such a collection of higher-dimensional simplices are called **cofaces**; the cofaces of a 1 -simplex $s = \{v_1, v_2\}$ could be something like $\{v_1, v_2, v_3\}$, $\{v_1, v_2, v_4\}$, $\{v_1, v_2, v_3, v_4\}$. More examples are shown in Fig. 4.

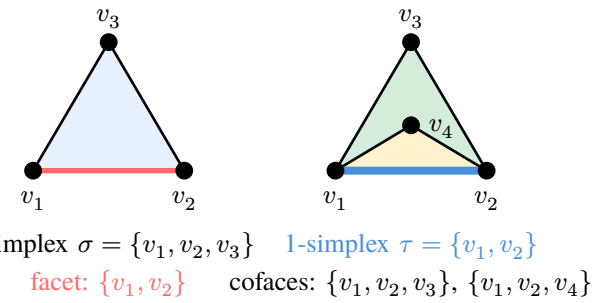


Fig. 4. Left: A facet of the 2-simplex $\{v_1, v_2, v_3\}$ is the edge $\{v_1, v_2\}$ (highlighted in red). Right: Cofaces of the 1-simplex $\{v_1, v_2\}$ (highlighted in blue) are the 2-simplices containing it.

These structures support natural memory operations. Coface lookup retrieves higher-dimensional simplices containing a query, identifying stronger contextual and narrative associations. Facet traversal descends to lower-dimensional faces, broadening the search when specificity must be relaxed.

When an agent retrieves knowledge relevant to a query, the retrieved simplices form a **subcomplex**, a simplicial complex $K' = (V', S')$ satisfying $V' \subseteq V$ and $S' \subseteq S$. Likewise, when a chat session is processed into text chunks, it creates a subcomplex, where each text chunk is a simplex. Then, by downward closure, we insert all cascading facets of the simplex, up to the 1-simplex. A chat session extracted to two text chunks with entities $\{v_1, v_2, v_3\}$ and $\{v_3, v_4\}$, for example, produces the subcomplex $\{v_1, v_2, v_3\}$, $\{v_3, v_4\}$ with cascading facets $\{v_1, v_2\}$, $\{v_2, v_3\}$, $\{v_1, v_3\}$. Efficient storage and retrieval of these subcomplexes: supporting insertion, membership queries, and coface lookup—requires a data structure designed for simplicial complexes.

IV. Simplex Trees

The simplex tree, introduced by Boissonnat and Maria in 2014, provides an efficient data structure for representing abstract simplicial complexes of arbitrary dimension[5]. The simplex tree reconciles the need to explicitly store all faces of the complex with the desire for compact representation and efficient operations, making it particularly well-suited for database-backed memory systems.

For the simplicial complex $K = (V, S)$ of dimension k (that is, the dimension of the largest simplex in K), we label each vertice $v_i \in V$ a letter $l_i \in \{1, ..., |V|\}$ from the alphabet $1, ..., |V|$, where $1 \leq l_1 < ... < l_{|V|} \leq |V|$. Then, the simplex $s = \{v_1, ..., v_i\}$ can be represented as the word $[s] = [l_1, ..., l_j]$. This allows us to express every k-simplex $s \in S$ as a word of $(k+1)$ length with labels of ascending order. We start with an empty **trie**, and construct the tree as such (Fig. 5):

- Words are inserted from the root or node to the leaf of the tree, with the first letter as the root or node and the last letter as the leaf.
- If inserting a word $[s] = [l_1, ...l_j]$, and the longest prefix word already in the tree is $\{l_1, ..., l_i\}$, of which $i < j$, we append the rest of the word $[l_{i+1}, ..., l_j]$ to the l_i node.

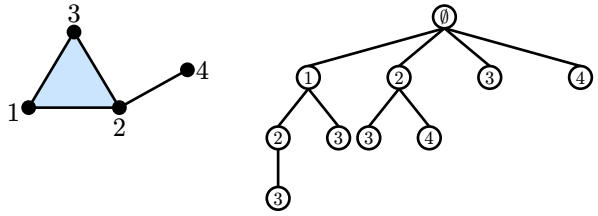


Fig. 5. A simplicial complex K with 2-simplex $\{1, 2, 3\}$ and edge $\{2, 4\}$ (left) and its simplex tree representation (right). Each path from root to node encodes a simplex: e.g., $\emptyset \to 1 \to 2 \to 3$ represents $\{1, 2, 3\}$.

In the original Boissonnat and Maria paper, the simplex tree was implemented as an in-memory structure. While in-memory databases could host these structures directly, they are unsuitable for persistent agent memory: RAM costs more per gigabyte than SSD storage, and requires continuous power to retain state. In-memory stores excel as caches, not as primary memory layers that must persist across sessions and scale with conversation history. We therefore implement the simplex tree using disk-backed persistence. Translating the requirements of the in-memory tree required some unique considerations:

1) An in-memory simplex tree uses red-black trees or hash tables for the trie structure, with the top nodes being an array. However, red-black trees are not used as database indexes due to being optimised for in-memory operations rather than disk-based operations[6]. Hash indexes are thus used in place, preserving the original time complexity of operations in the paper. A breakdown of the time complexity of operations in the persistent agent memory is given in Table I.

TABLE I
TIME COMPLEXITY OF SIMPLEX TREE OPERATIONS.

Operation	**Purpose**	**Complexity**
Insert simplex	Record observed co-occurrence from extraction	$O(j)$
Membership check	Verify whether a face exists during gap detection	$O(j)$
Cofaces of vertex set	Find all simplices containing query-matched vertices	$O\left(kT^{>0}_{\text{last}(v)}\right)$
Enumerate theoretical faces	Compute expected faces for filtration comparison	$O(2^j)$
Delete simplex	Memory management, forgetting	$O(j)$

Here, j is the simplex dimension, k is the dimension of the simplicial complex, and $T^{>0}_{\text{last}(s)}$ is the total number of nodes in the simplex tree storing $\text{last}(s)$, the last letter of the simplex s, at a tree depth greater than 0. In practice, this value is bounded by $C_{\{\text{last}(s)\}}$, the number of cofaces containing the last letter of the simplex.

2) In the in-memory implementation, each sibling node has a pointer to its parent node, and for all nodes in the same tree depth of the same letter label l_i, they are connected in a circular linked list. Both cases are achievable in the relational model by using foreign-key primary-key relations, which can be shown in greater detail in the next section.

V. DATABASE DESIGN

We translate simplex trees to persistent database schemas with three core tables, adapting standard approaches to modelling graphs in relational models[7]. The full schema definition is provided in the Appendix.

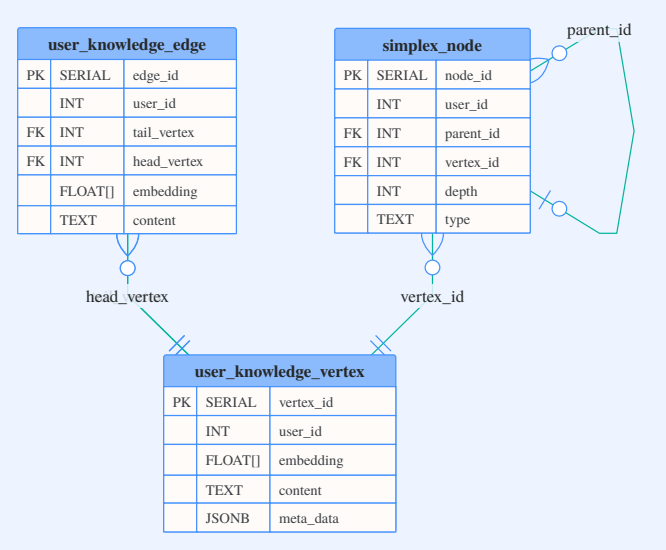


Fig. 6. Entity-relationship diagram for the persistence layer. The `simplex_node` table implements the trie structure with a self-referential foreign key for parent traversal. Both `user_knowledge_edge` and `simplex_node` reference vertices in `user_knowledge_vertex`.

VI. Appendix

A. Database Schema Definition

```
CREATE TABLE user_knowledge_vertex (
    vertex_id    SERIAL PRIMARY KEY,
    user_id      INT NOT NULL,
    embedding    FLOAT[] NOT NULL,
    content      TEXT NOT NULL,
    meta_data    JSONB DEFAULT '{}'
);

CREATE TABLE user_knowledge_edge (
    edge_id      SERIAL PRIMARY KEY,
    user_id      INT NOT NULL,
    tail_vertex INT REFERENCES
user_knowledge_vertex(vertex_id),
    head_vertex INT REFERENCES
user_knowledge_vertex(vertex_id),
    embedding    FLOAT[],
    content      TEXT,
    meta_data    JSONB DEFAULT '{}'
);

CREATE TABLE simplex_node (
    node_id      SERIAL PRIMARY KEY,
    user_id      INT NOT NULL,
    parent_id    INT REFERENCES
simplex_node(node_id),
    vertex_id    INT REFERENCES
user_knowledge_vertex(vertex_id),
    depth        INT NOT NULL,
    type         TEXT,
    meta_data    JSONB DEFAULT '{}'
);

-- Unique constraint handling NULL
parent_id for root nodes
CREATE UNIQUE INDEX idx_simplex_unique
    ON simplex_node (user_id, parent_id,
vertex_id)
    WHERE parent_id IS NOT NULL;
CREATE UNIQUE INDEX
idx_simplex_root_unique
    ON simplex_node (user_id, vertex_id)
    WHERE parent_id IS NULL;

-- Child lookup (hash for O(1) expected)
CREATE INDEX idx_children
    ON simplex_node USING HASH
(parent_id);

-- L_j(ℓ) lists: find all nodes
referencing a vertex at given depth
CREATE INDEX idx_vertex_depth
    ON simplex_node (user_id, vertex_id,
depth);

-- Parent traversal
CREATE INDEX idx_parent
    ON simplex_node (parent_id);
```

References

[1] Supermemory.ai, "Supermemory is the new State-of-the-Art in agent memory." [Online]. Available: https://supermemory.ai/research

[2] C. Latimer *et al.*, "Hindsight is 20/20: Building Agent Memory that Retains, Recalls, and Reflects," *arXiv preprint arXiv:2512.12818*, 2025, doi: 10.48550/arxiv.2512.12818.

[3] Emergence.ai, "SOTA on LongMemEval with RAG." [Online]. Available: https://www.emergence.ai/blog/sota-on-longmemeval-with-rag

[4] V. Salnikov, D. Cassese, R. Lambiotte, and N. Jones, "Co-occurrence simplicial complexes in mathematics: identifying the holes of knowledge," *Applied Network Science*, vol. 3, no. 1, 2018, doi: 10.1007/s41109-018-0074-3.

[5] J.-D. Boissonnat and C. Maria, "The Simplex Tree: an Efficient Data Structure for General Simplicial Complexes," *arXiv preprint arXiv:2001.02581*, 2020, doi: 10.48550/arxiv.2001.02581.

[6] C. Du, "Why MySQL Uses B+Tree As The Index?." [Online]. Available: https://chaodu.hashnode.dev/why-mysql-uses-btree-as-the-index

[7] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media, 2018.